



Faculty of Engineering – Cairo University
Electronics and Electrical Communications
ELC 3030 – Advanced Microprocessors



Advanced Processor Architecture - Project

Submitted to:

Eng. Hassan El-Menier

ID	Name
9230557	عبدالله حمدي احمد رمضان
9230628	عمرو الحسن يعقوب حامد
9230565	عبدالله مصطفى محمد مصطفى
9230717	مازن محمد رفاعي صالح
9230556	عبدالله تامر محمد عبدالصمد

Contents

1 Introduction	4
2 Overall Processor Architecture	4
3 Top Module Description	5
Top Module Interfaces	5
4 Fetch Stage	6
Main Functions	6
Program Counter Logic	6
Outputs	6
5 Decode Stage	6
Instruction Decoding	6
Register File	6
Control Signal Generation	7
Conditional Branching	7
6 Forwarding Unit (Data Hazard Resolution)	7
Forwarding Sources	7
7 Execute Stage	8
7.1 Execute Stage Responsibilities	8
7.2 ALU Operations	8
7.3 Pipeline Registers	9
7.4 Stack and Control Flow Instructions	9
7.5 Output Port Handling	9
8 Memory Stage	10
8.1 Memory Organization	10
8.2 Memory Operations	10
8.3 Write Back Logic	10
8.4 Program Counter Redirection	10
8.5 Integration with Write Back	10
9 Instruction Execution Flow	11
9.1 Example 1: Arithmetic Instruction (ADD)	11
Instruction	11
Cycle-by-Cycle Execution	11
9.2 Example 2: Load Instruction (LDD)	12
Instruction	12
Pipeline Flow	12
9.3 Example 3: Store Instruction (STD)	12
Instruction	12
Pipeline Flow	12

9.4 Example 4: Control Flow Instruction (CALL)	13
Instruction.....	13
Pipeline Flow	13
9.5 Example 5: Interrupt Handling (RTI).....	13
Instruction.....	13
Pipeline Flow	14
9.6 Data Hazard Resolution (Forwarding Example)	14
Scenario	14
Solution	14
9.7 Summary.....	14
10. Simulation & Testbench Verification	15
10.1 Testbench Script.....	15
10.2 Testbench Overview	16
10.3 Test Program Description	16
Instruction 1: LDM R0, #10.....	17
Instruction 2: LDM R1, #30.....	17
Instruction 3: ADD R0, R1.....	18
Instruction 4: SUB R0, R1.....	18
Instruction 5: OUT R0	19
Instruction 6: CALL R1.....	19
Instruction 7: STD R0, 100.....	20
Instruction 8 RET.....	20
11. Synthesis and Implementation Results	21
11.1 RTL Schematic	21
Explanation.....	22
11.2 Synthesis Schematic	22
Explanation.....	24
11.3 Target FPGA Platform	24
11.4 Timing Analysis – Synthesis	25
Explanation.....	25
11.5 Timing Analysis – FPGA Implementation	25
Explanation.....	25
11.6 Frequency Calculation.....	26
Formula Used.....	26
At Synthesis Stage	26
At FPGA Implementation.....	26
11.7 Why Is the Frequency Different?.....	26
11.8 FPGA Utilization	27
Explanation.....	27

11.9 Discussion.....	27
11.10 Limitations and Challenges.....	27
11.11 Future Work	27
11.12 Conclusion	27
11.13 Verification Summary.....	28
Appendix	29

1. Introduction

This project presents the design and implementation of an 8-bit pipelined processor using Verilog HDL. The processor follows a RISC-like architecture and supports a wide range of instructions including arithmetic, logical, memory access, branching, looping, stack operations, and interrupt handling.

The processor is divided into multiple pipeline stages to improve instruction throughput. Data hazards are resolved using a forwarding mechanism, while control hazards are handled through program counter redirection logic. The design was simulated and synthesized using Xilinx Vivado to evaluate functionality, performance, and FPGA resource utilization.

2. Overall Processor Architecture

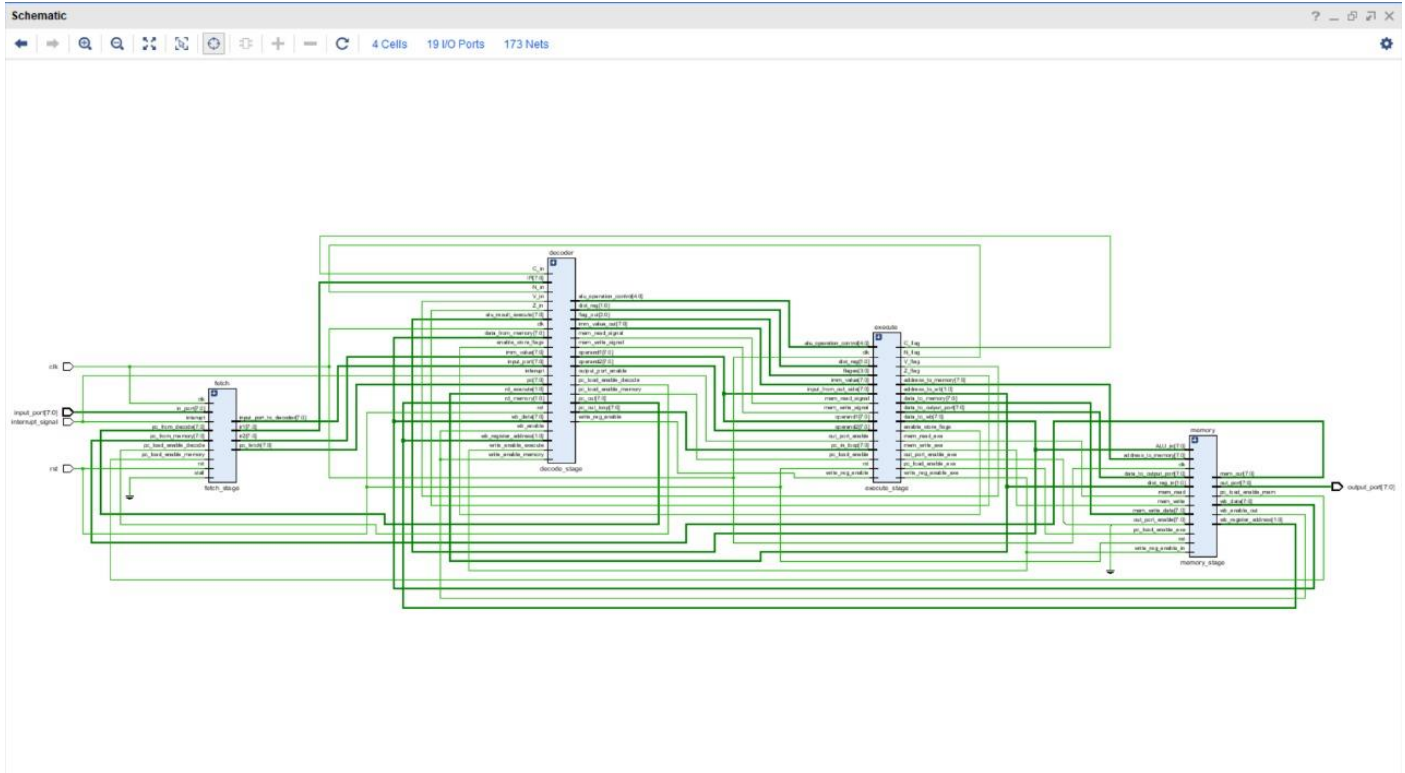
The processor is organized as a **five-stage pipeline** consisting of:

1. Fetch Stage
2. Decode Stage
3. Execute Stage
4. Memory Stage
5. Write Back (integrated within Memory stage)

The architecture follows a **Harvard model**, where instructions and data share the same memory space. The processor supports hardware interrupts, stack-based subroutine calls, and conditional branching using status flags.

Key architectural features include:

- 8-bit data path
- Four general-purpose registers (R0–R3), where R3 acts as the Stack Pointer (SP)
- Hardware interrupt support with context saving
- Forwarding unit for resolving data hazards
- Unified memory for instructions and data



3. Top Module Description

The Top module integrates all processor stages and manages signal flow between them. It acts as the main wrapper that connects the pipeline stages together and interfaces with the external environment.

Top Module Interfaces

- **Inputs:**
 - **clk:** System clock
 - **rst:** Active-low reset
 - **input_port:** External input data
 - **interrupt_signal:** Hardware interrupt request
- **Outputs:**
 - **output_port:** External output data

The top module instantiates the following submodules:

- **fetch_stage**
- **decode_stage**
- **execute_stage**
- **memory_stage**

Pipeline control signals, forwarding signals, write-back paths, and program counter control signals are routed through this module to ensure correct execution across pipeline stages.

4. Fetch Stage

The Fetch stage is responsible for retrieving instructions from instruction memory and calculating the next value of the Program Counter (PC).

Main Functions

- Instruction fetch from instruction memory
- Program Counter update
- Interrupt handling
- Control hazard handling (jumps, calls, returns)
- Support for variable-length instructions

Program Counter Logic

The next PC value is selected based on the following priority:

1. **Interrupt handling:** Jump to interrupt service routine vector
2. **Pipeline stall:** Freeze PC
3. **Return instructions (RET / RTI):** PC loaded from memory stage
4. **Jump and branch instructions:** PC loaded from decode stage
5. **Normal execution:** PC increment

For two-byte instructions such as **LDM**, **LDD**, and **STD**, the **PC** is incremented by 2. Otherwise, the PC increments by 1.

Outputs

- **pc_fetch:** Current PC value forwarded to Decode stage
- **ir1:** Instruction opcode
- **ir2:** Immediate value or next instruction byte
- **input_port_to_decoder:** Forwarded external input

5. Decode Stage

The Decode stage performs instruction decoding, operand fetching, control signal generation, and hazard resolution.

Instruction Decoding

The instruction opcode is extracted from bits [7:4], while register addresses are extracted from bits [3:2] and [1:0].

Register File

- Four 8-bit registers (R0–R3)

- R3 is dedicated as the Stack Pointer (SP)
 - Write-back is handled from the Memory stage
-

Control Signal Generation

Based on the decoded opcode, the Decode stage generates:

- ALU operation control signals
 - Memory read/write signals
 - Register write enable
 - Program counter control signals
 - Output port enable
-

Conditional Branching

Conditional branch instructions (JZ, JN, JC, JV) are resolved using stored status flags:

- Zero (Z)
- Negative (N)
- Carry (C)
- Overflow (V)

If the condition is satisfied, the PC is updated accordingly.

6. Forwarding Unit (Data Hazard Resolution)

To resolve data hazards without stalling the pipeline, a **forwarding unit** is implemented in the Decode stage.

Forwarding Sources

- Execute stage (highest priority)
- Memory stage

The forwarding unit compares source register addresses in the Decode stage with destination registers in the Execute and Memory stages. If a match is detected and the destination register is scheduled for write back, the most recent data is forwarded directly to the ALU operands.

This mechanism ensures correct execution of dependent instructions and improves overall pipeline performance.

7. Execute Stage

The Execute stage performs all arithmetic, logical, control-flow, and address calculation operations. It is considered the core computational stage of the processor and contains the ALU, flag logic, and pipeline registers.

7.1 Execute Stage Responsibilities

- Perform ALU operations (ADD, SUB, AND, OR, shifts, etc.)
- Update status flags (Z, N, C, V)
- Generate memory addresses for load/store instructions
- Handle stack operations (PUSH, POP, CALL, RET, RTI)
- Control output port operations
- Pass results and control signals to the Memory stage

7.2 ALU Operations

The ALU behavior is controlled using the `alu_operation_control` signal generated by the Decode stage. Each instruction maps to a specific ALU operation code.

Supported ALU operations include:

- Arithmetic: ADD, SUB, INC, DEC, NEG
- Logical: AND, OR, NOT
- Shift/Rotate: RLC, RRC
- Control-related operations: CALL, RET, RTI, LOOP
- Immediate operations: LDM, LDD, STD, LDI, STI

The ALU produces:

- **ALU_out**: Result of the computation
- Updated status flags:
 - **Z (Zero)**: Result equals zero
 - **N (Negative)**: MSB of result is 1
 - **C (Carry)**: Carry out from arithmetic operation
 - **V (Overflow)**: Signed overflow detection

Flags are stored conditionally using the `enable_store_flags` signal.

7.3 Pipeline Registers

To maintain correct pipeline operation, the Execute stage uses internal pipeline registers to delay incoming signals from Decode stage by one clock cycle. These include:

- Operands (`operand1_delay`, `operand2_delay`)
- Control signals (`mem_read_signal_delay`, `mem_write_signal_delay`)
- Destination register (`dest_reg_delay`)
- ALU operation control
- Immediate values
- PC control signals

This ensures proper synchronization between pipeline stages.

7.4 Stack and Control Flow Instructions

The Execute stage handles stack-based instructions using register R3 as the Stack Pointer (SP).

Examples:

- **PUSH:** Decrements SP and writes data to memory
- **POP:** Reads data from memory and increments SP
- **CALL:** Pushes return address onto stack and updates PC
- **RET / RTI:** Pops return address from stack and redirects PC

For RET and RTI instructions, the Execute stage activates the `pc_load_enable_exe` signal to notify the Memory stage that the PC should be updated.

7.5 Output Port Handling

For the `OUT` instruction, data is routed to the external output port using:

- `data_to_output_port`
- `out_port_enable_exe`

This separates I/O operations from normal register write-back logic.

8. Memory Stage

The Memory stage handles all data memory operations and final write-back to the register file. It also plays a role in PC redirection for return instructions.

8.1 Memory Organization

- Data memory size: **128 bytes**
- 8-bit wide memory entries
- Shared memory space for stack and data operations

The memory is modeled using a Verilog array and is synchronous with the system clock.

8.2 Memory Operations

The Memory stage supports:

- **Memory Write:** For instructions such as STD, STI, PUSH, CALL
- **Memory Read:** For instructions such as LDD, LDI, POP, RET

When `mem_write` is asserted, data is written to memory at the specified address.

When `mem_read` is asserted, data is read from memory and forwarded to the write-back path.

8.3 Write Back Logic

The Memory stage determines the source of data written back to the register file:

1. **Memory Read Instructions**
 - Data is read from memory and assigned to `wb_data`
2. **ALU Instructions**
 - ALU result is passed directly to write-back
3. **Output Instructions**
 - Data is routed to the external output port

The destination register and write enable signal are passed through from the Execute stage.

8.4 Program Counter Redirection

For control-flow instructions such as **RET** and **RTI**, the Memory stage retrieves the return address from memory and outputs it through `mem_out`.

The `pc_load_enable_mem` signal informs the Fetch stage to load the new PC value.

This design ensures correct pipeline redirection without stalling.

8.5 Integration with Write Back

The write-back operation is integrated within the Memory stage rather than being implemented as a separate pipeline stage. This simplifies control logic while maintaining correct pipeline behavior.

9. Instruction Execution Flow

This section explains how instructions propagate through the processor pipeline stages. The processor follows a **five-stage pipelined architecture**:

1. Fetch
2. Decode
3. Execute
4. Memory
5. Write Back

Each instruction advances one stage per clock cycle, allowing multiple instructions to be processed simultaneously.

9.1 Example 1: Arithmetic Instruction (ADD)

Instruction

ADD R1, R2

(Add contents of R2 to R1 and store result in R1)

Cycle-by-Cycle Execution

➤ *Cycle 1 – Fetch*

- The instruction is fetched from instruction memory using **PC**.
- **IR** is loaded with the opcode.
- **PC** is incremented automatically.

➤ *Cycle 2 – Decode*

- Opcode is decoded as **ADD**.
- Source registers:
 - R1 → operand1
 - R2 → operand2
- Control signals generated:
 - **alu_operation_control** = **ADD**
 - **write_reg_enable** = 1
- Forwarding logic checks for data hazards.

➤ *Cycle 3 – Execute*

- ALU performs addition:
- **ALU_out** = **operand1** + **operand2**
- Flags updated:
 - Z, N, C, V
- Result forwarded to the Memory stage.

➤ *Cycle 4 – Memory*

- No memory operation is required.
- ALU result is passed directly to write-back logic.

➤ *Cycle 5 – Write Back*

- Result is written into register R1.
- **wb_enable = 1**

9.2 Example 2: Load Instruction (LDD)

Instruction

LDD R1, #0x20

Pipeline Flow

➤ *Fetch*

- Instruction and immediate value are fetched (2-byte instruction).

➤ *Decode*

- Immediate value extracted.
- Control signals:
 - **mem_read_signal = 1**
 - **write_reg_enable = 1**
- Target register = R1.

➤ *Execute*

- Address calculation:
- **address_to_memory = immediate value**

➤ *Memory*

- Data read from memory at address **0x20**.

➤ *Write Back*

- Loaded data written to R1.
-

9.3 Example 3: Store Instruction (STD)

Instruction

STD R2, #0x30

Pipeline Flow

➤ *Decode*

- Operand fetched from R2.
- Immediate value decoded as memory address.

➤ *Execute*

- Address and data prepared.

➤ *Memory*

- Data from R2 is written to memory at address **0x30**.

➤ *Write Back*

- No write-back operation (store instruction).
-

9.4 Example 4: Control Flow Instruction (CALL)

Instruction

CALL R2

Pipeline Flow

➤ *Decode*

- Stack pointer (R3) selected.
- Return address (**PC + 1**) calculated.
- Control signals:
 - **mem_write_signal = 1**
 - **pc_load_enable = 1**

➤ *Execute*

- Return address pushed onto stack.
- Stack pointer decremented.

➤ *Memory*

- Return address written to memory.
- PC updated with target address from R2.

➤ *Fetch*

- Execution resumes from the called subroutine.
-

9.5 Example 5: Interrupt Handling (RTI)

Instruction

RTI

Pipeline Flow

➤ *Interrupt Detection*

- Interrupt signal forces PC to jump to ISR.

➤ *Execute*

- Flags and return address restored.

➤ *Memory*

- PC is loaded from memory.
- Flags restored.

9.6 Data Hazard Resolution (Forwarding Example)

Scenario

ADD R1, R2

SUB R3, R1

Without forwarding, **SUB** would use an incorrect value of R1.

Solution

- Forwarding unit detects dependency.
- Result from Execute stage is forwarded directly to Decode stage.
- No pipeline stall is required.

9.7 Summary

- The pipeline allows one instruction per clock cycle after fill-up.
- Forwarding minimizes stalls and improves performance.
- Control hazards are resolved using PC load enable signals.
- Memory and write-back operations are correctly synchronized.

10. Simulation & Testbench Verification

10.1 Testbench Script

```
`timescale 1ns / 1ps

module top_tb;
    reg clk;
    reg rst;
    reg [7:0] input_port;
    reg interrupt_signal;
    wire [7:0] output_port;

    // Instantiate the Top module
    top dut (
        .clk(clk),
        .rst(rst),
        .input_port(input_port),
        .interrupt_signal(interrupt_signal),
        .output_port(output_port)
    );

    // Clock Generation
    always #5 clk = ~clk;

    initial begin
        // Initialize signals
        clk = 0;
        rst = 0;
        input_port = 8'h00;
        interrupt_signal = 0;

        // Reset the system
        #10 rst = 1;

        // --- Load Instructions into Fetch Memory ---
        // Note: Replace dut.fetch.instruction_memory with the correct path if needed

        // 1. LDM R0, 10 (Load 10 into R0) -> 2-byte instruction
        dut.fetch.instruction_memory[1] = 8'hC0; // Opcode 12, rb=0
        dut.fetch.instruction_memory[2] = 8'h0A; // Imm = 10

        // 2. LDM R1, 30 (Load 30 into R1)
        dut.fetch.instruction_memory[3] = 8'hC1; // Opcode 12, rb=1
        dut.fetch.instruction_memory[4] = 8'h1E; // Imm = 30
        // 3. ADD R0, R1 (R0 = 10 + 30 = 40)
        dut.fetch.instruction_memory[5] = 8'h21; // Opcode 2, ra=0, rb=1

        // 4. SUB R0, R1 (R0 = 40 - 30 = 10)
        dut.fetch.instruction_memory[6] = 8'h31; // Opcode 3, ra=0, rb=1
    end
endmodule
```



```

// 5. OUT R0 (Send R0 to output_port)
dut.fetch.instruction_memory[7] = 8'h78; // Opcode 7, ra=2, rb=0

// 6. CALL R1 (Jump to address in R1, which is 5)
// This tests the Control Hazard (flushing/stalling)
dut.fetch.instruction_memory[8] = 8'hB5; // Opcode 11, brx=1, rb=1
// 7. STD R0, 100 (Store R0 at Mem[100]) - At subroutine address
dut.fetch.instruction_memory[30] = 8'hC8; // Opcode 12, rb=0
dut.fetch.instruction_memory[31] = 8'h64; // ea = 100

// 8. RET (Return to address 8)
dut.fetch.instruction_memory[32] = 8'hB8; // Opcode 11, brx=2

// Wait for execution
#200;
$stop;
end
endmodule

```

10.2 Testbench Overview

The testbench is designed to verify the correct functionality of the processor by executing a small program that covers:

- Arithmetic operations
- Memory operations
- I/O operations
- Control flow instructions (CALL / RET)
- Pipeline behavior

A clock with a period of **10 ns** is generated, and the processor is reset before instruction execution begins.

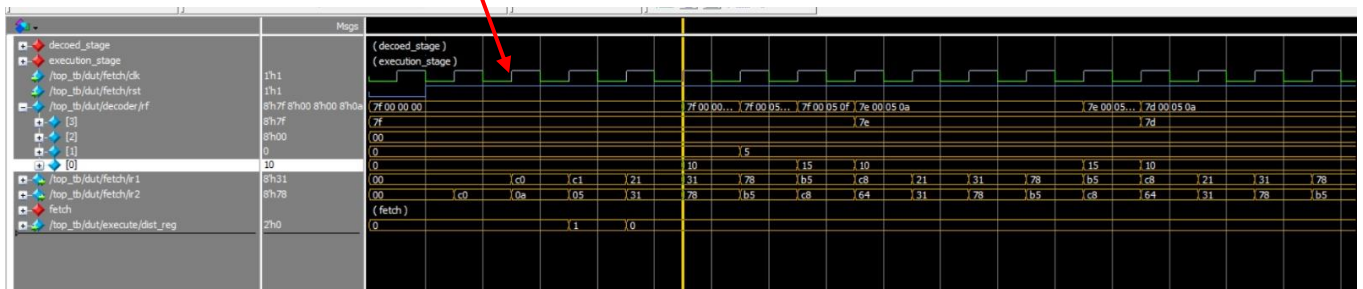
10.3 Test Program Description

The following instructions are loaded manually into the instruction memory:

Description	Instruction	Address
Load immediate value 10 into R0	LDM R0, 10	1–2
Load immediate value 30 into R1	LDM R1, 30	3–4
$R0 = 10 + 30 = 40$	ADD R0, R1	5
$R0 = 40 - 30 = 10$	SUB R0, R1	6
Send R0 to output port	OUT R0	7
Jump to subroutine at address in R1	CALL R1	8
Store R0 in memory	STD R0, 100	30-31
Return to main program	RET	32

Instruction 1: LDM R0, #10

End of fetch stage



Explanation

At this time interval, the instruction **LDM R0, #10** is fetched.

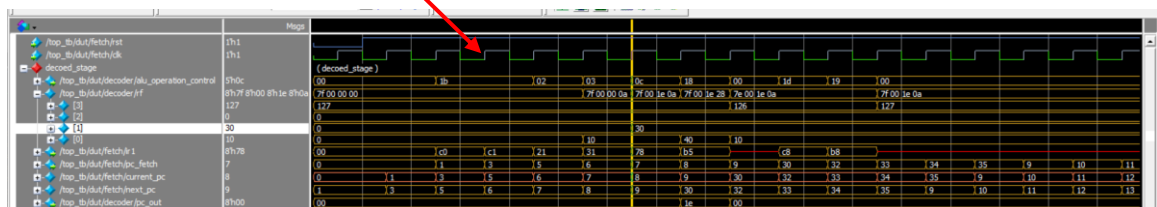
Since **LDM** is a two-byte instruction, the program counter increments by 2.

The immediate value is passed directly to the Execute stage and then written back to register R0.

The write-back enable signal is asserted, confirming successful register update.

Instruction 2: LDM R1, #30

End of fetch stage



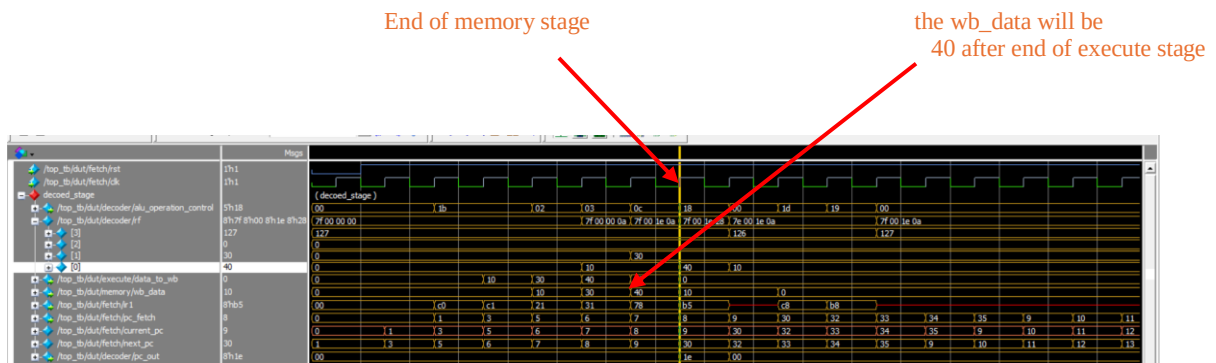
Explanation

The second instruction loads the immediate value 30 into register R1.

Similar to the previous instruction, the PC increments by 2.

The write-back stage correctly updates register R1 with the immediate value.

Instruction 3: ADD R0, R1



Explanation

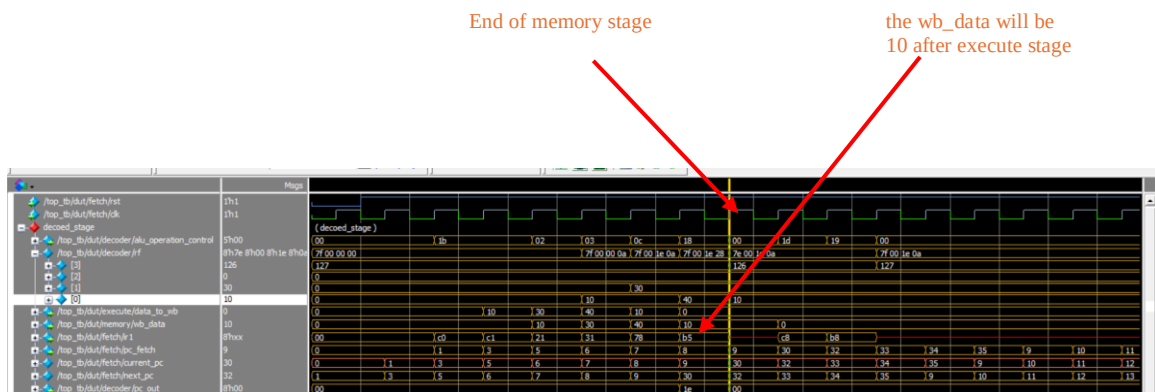
The ADD instruction adds the contents of R0 and R1.

The ALU computes the result (40), which is forwarded directly to the write-back stage.

The condition flags are updated based on the ALU result.

Forwarding is used to avoid pipeline stalls.

Instruction 4: SUB R0, R1



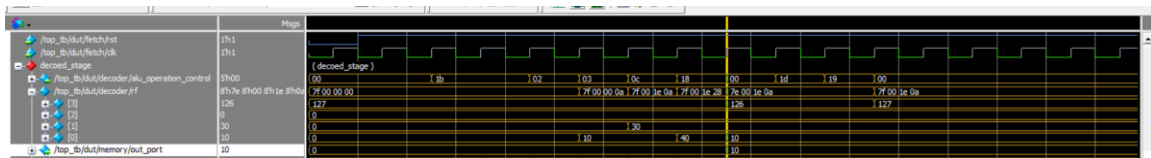
Explanation

The SUB instruction subtracts R1 from R0.

The ALU produces the correct result (10), which is written back to R0.

The flags are updated accordingly, confirming correct arithmetic behavior.

Instruction 5: OUT R0



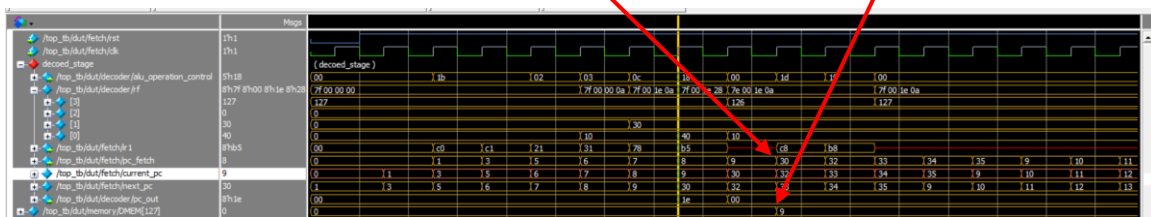
Explanation

The OUT instruction sends the contents of R0 to the output port.
The output enable signal is asserted, and the output port reflects the expected value (10).

Instruction 6: CALL R1

the pc_fetch will be updated by the value in R1
will be 30

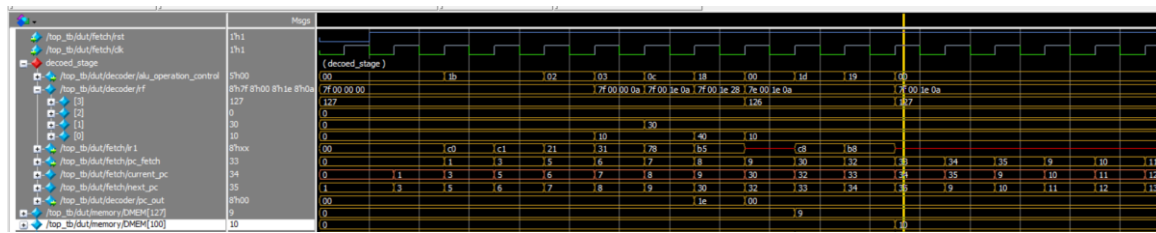
The Pc_fetch +1 will be pushed at the memory
(Pc_fetch = 8)



Explanation

During the CALL instruction, the return address is pushed onto the stack.
The stack pointer is decremented, and the PC is updated with the target address stored in R1.
This demonstrates correct handling of control hazards.

Instruction 7: STD R0, 100



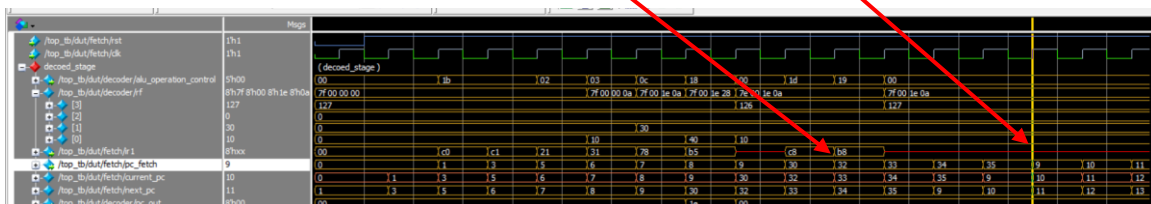
Explanation

The STD instruction stores the value of R0 into data memory at address 100. The memory write enable signal is asserted, and the correct data is written to memory.

Instruction 8 RET

the end of fetch interrupt

the pc_fetch will be returned from mem

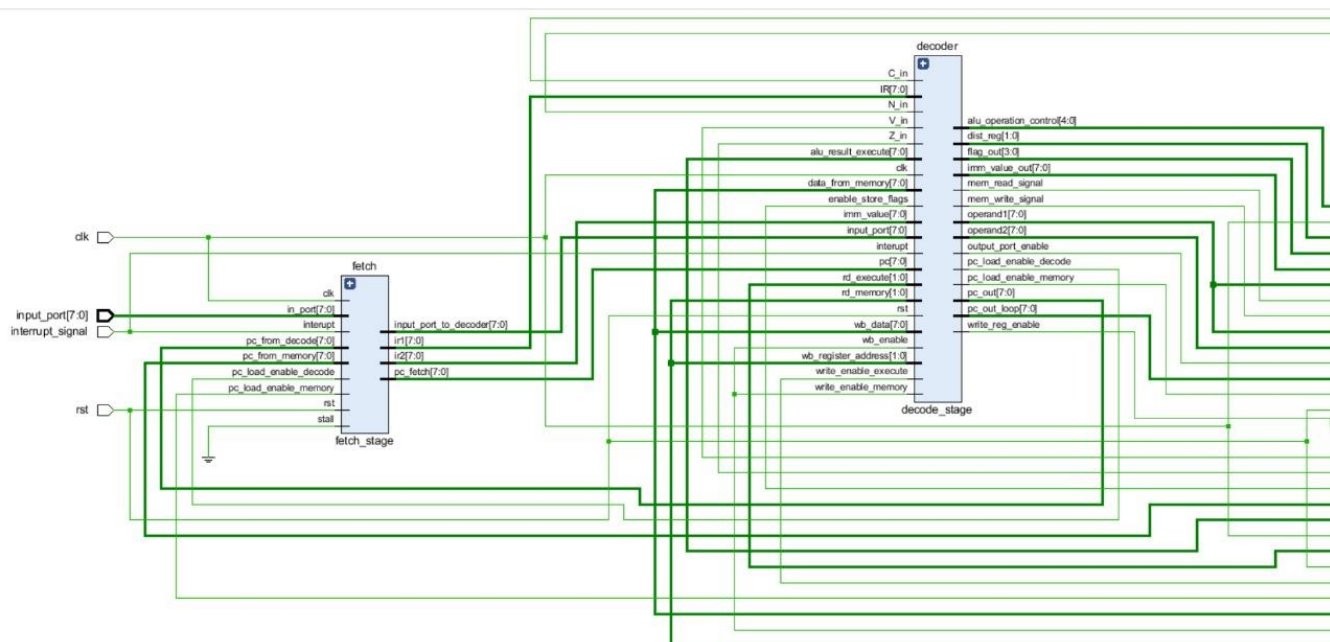
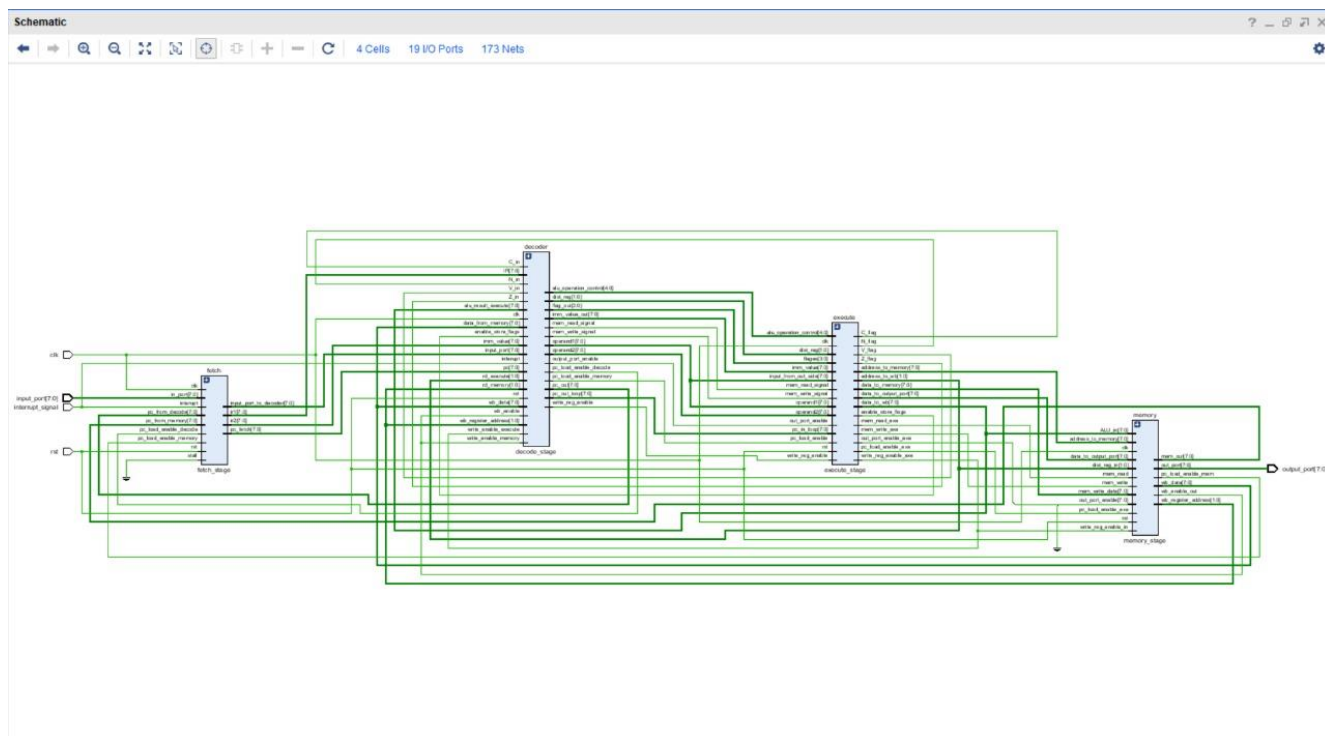


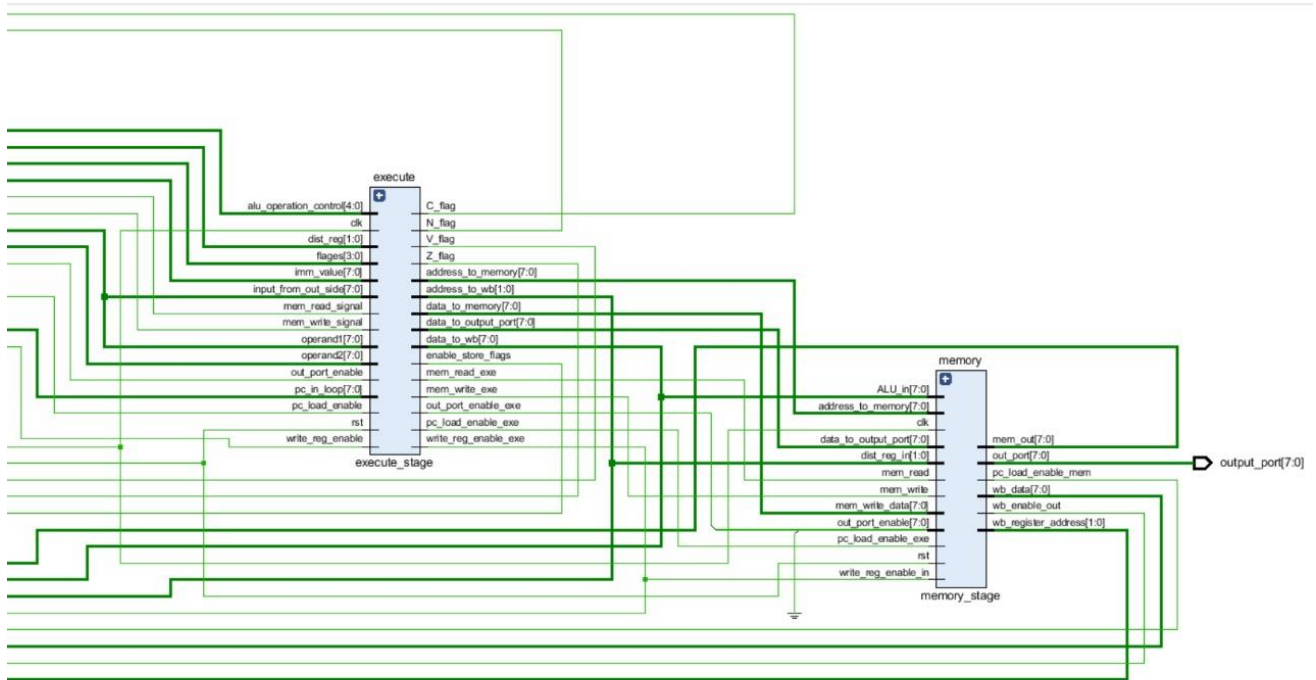
Explanation

The RET instruction reads the return address from the stack and updates the PC accordingly. Program execution resumes from the correct instruction after the CALL.

11. Synthesis and Implementation Results

11.1 RTL Schematic





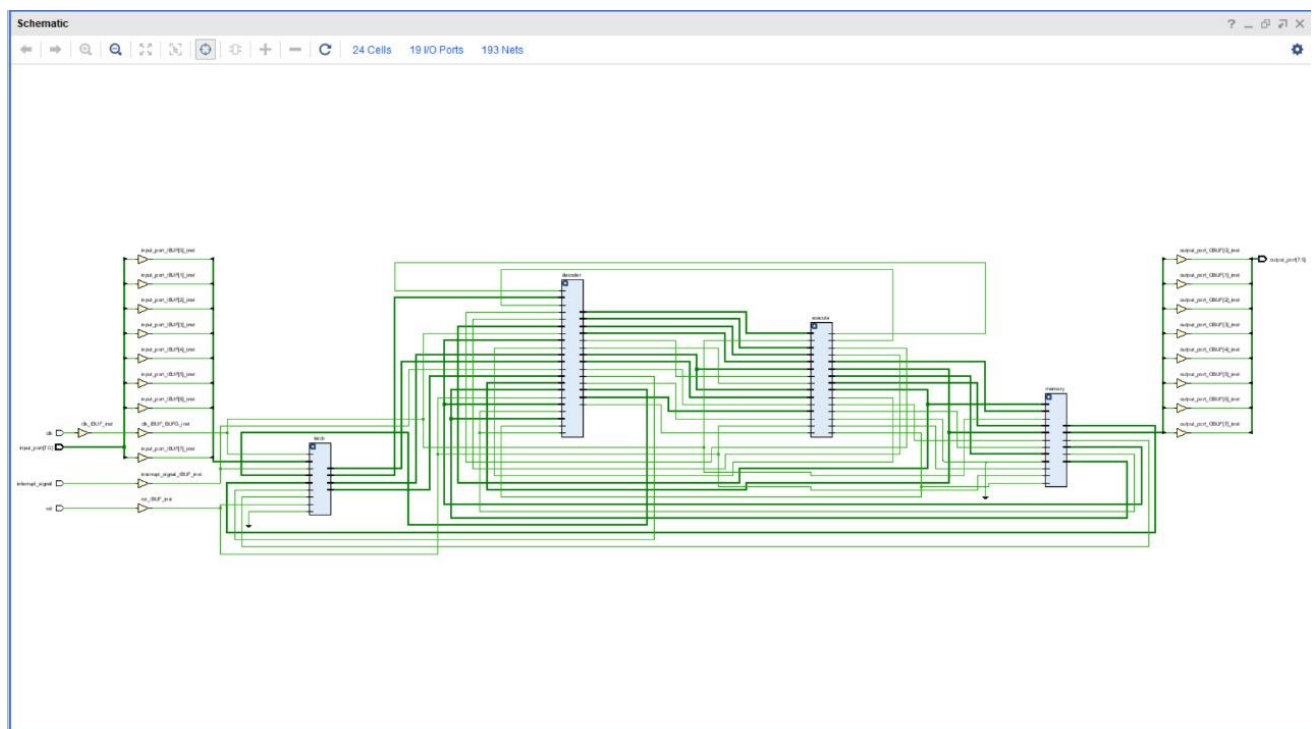
Explanation

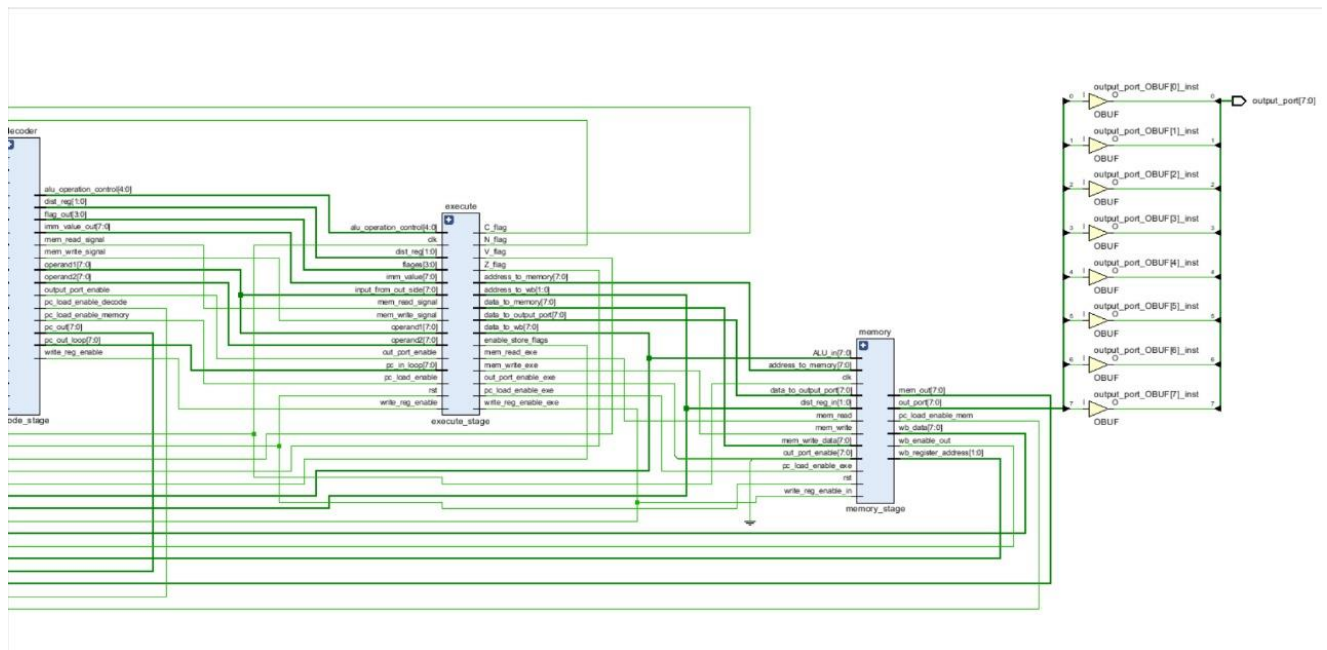
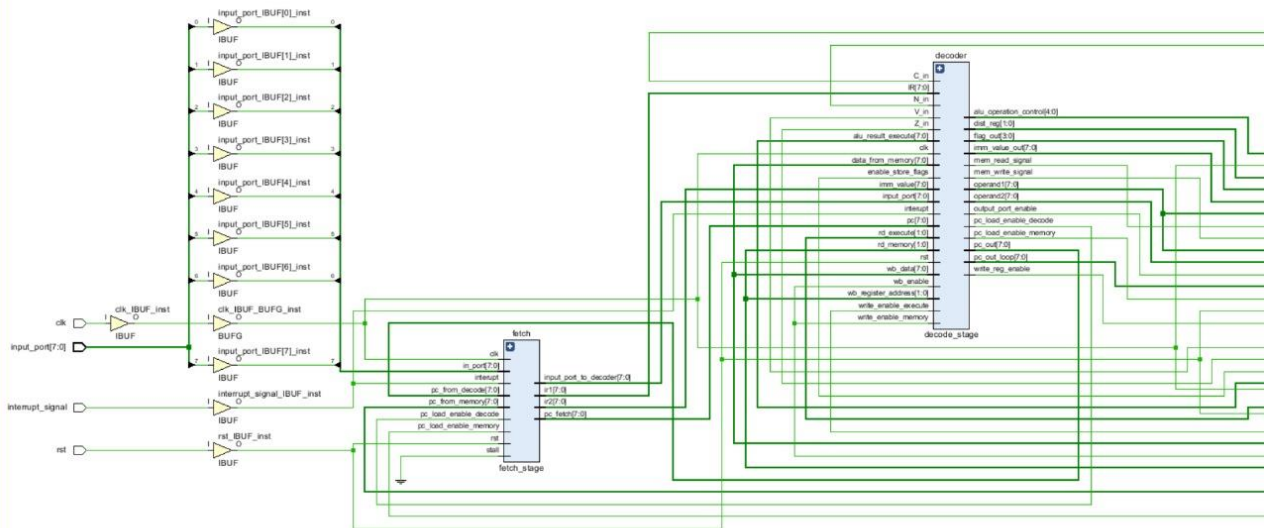
The RTL schematic illustrates the structural design of the processor before synthesis.

It shows the main pipeline stages: Fetch, Decode, Execute, Memory, and Write Back, along with the control and data paths between them.

This representation confirms correct module integration and signal connectivity at the register-transfer level.

11.2 Synthesis Schematic



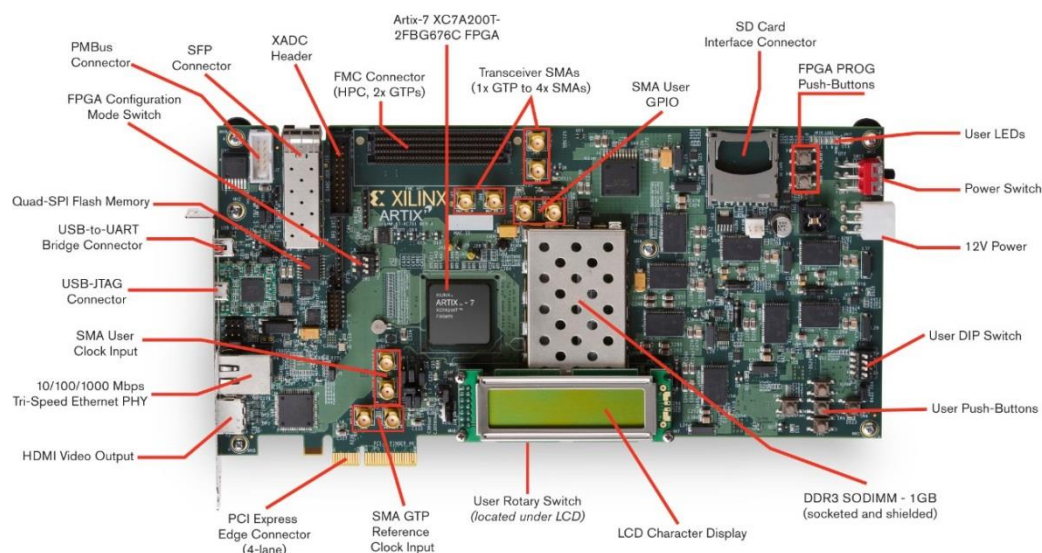


Explanation

The synthesis schematic represents the optimized hardware after logic synthesis. High-level behavioral constructs are mapped into FPGA primitives such as LUTs, flip-flops, and multiplexers.

Redundant logic is optimized, resulting in a more compact and efficient design.

11.3 Target FPGA Platform



The processor was synthesized and implemented targeting the **Artix-7 AC701 Evaluation Platform (xc7a200t-2fbg676-2)**.

This FPGA provides sufficient logic resources and high-speed interconnects suitable for implementing a pipelined processor architecture.

All synthesis, implementation, timing, and utilization results reported in this section correspond to this target device.

11.4 Timing Analysis – Synthesis

General Information	Setup	Hold	Pulse Width
Timer Settings Design Timing Summary	Worst Negative Slack (WNS): 0.186 ns	Worst Hold Slack (WHS): 0.145 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Clock Summary (1) no_clock (0) constant_clock (0)	Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
	Total Number of Endpoints: 2204	Total Number of Endpoints: 2204	Total Number of Endpoints: 1165
	All user specified timing constraints are met.		

Explanation

The synthesis timing report provides an estimate of the maximum achievable clock frequency based on logic delays only.

The Worst Negative Slack (WNS) at synthesis is **+0.186 ns**, indicating that the design meets the timing requirements with a positive margin.

11.5 Timing Analysis – FPGA Implementation

General Information	Setup	Hold	Pulse Width
Timer Settings Design Timing Summary	Worst Negative Slack (WNS): -0.272 ns	Worst Hold Slack (WHS): 0.163 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Clock Summary (1) Intra-Clock Paths clk Setup -0.272 ns (10)	Total Negative Slack (TNS): -4.247 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
	Number of Failing Endpoints: 72	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
	Total Number of Endpoints: 2204	Total Number of Endpoints: 2204	Total Number of Endpoints: 1165
	Timing constraints are not met.		

Explanation

After place-and-route, the timing analysis reflects real physical delays caused by routing and interconnects. The Worst Negative Slack (WNS) at FPGA implementation is **-0.272 ns**, and the total slack reaches **-4.247 ns**, indicating a timing violation.

Negative slack means the design cannot reliably operate at the specified clock frequency.

11.6 Frequency Calculation

Formula Used

The maximum operating frequency is calculated using the following approximation:

$$f_{max} = \frac{1}{T_{clk} + |WNS|}$$

At Synthesis Stage

- Worst Negative Slack (WNS) = **+0.186 ns**

$$f_{max} = \frac{1}{5\text{ ns} + 0.186\text{ ns}} = 192.8\text{ MHz}$$

Since WNS is positive, the design meets timing and can operate at a higher frequency than the target clock.

At FPGA Implementation

- Worst Negative Slack = **-0.272 ns**
- Total Negative Slack = **-4.247 ns**

$$f_{max} = \frac{1}{5\text{ ns} + 0.272\text{ ns}} = 189.6\text{ MHz}$$

The negative slack indicates that the critical path exceeds the clock period, reducing the achievable frequency.

11.7 Why Is the Frequency Different?

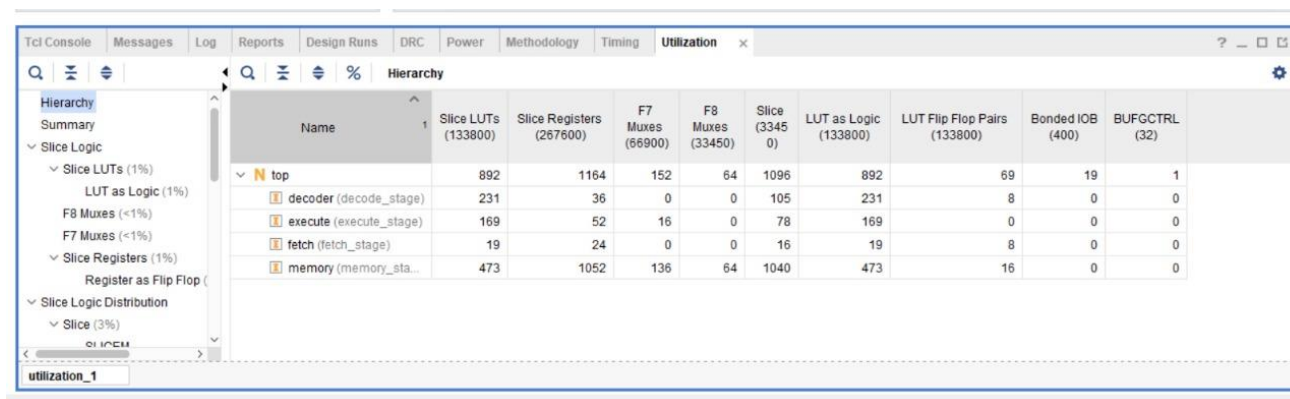
The operating frequency differs between synthesis and FPGA implementation due to physical effects that are not considered during synthesis.

During synthesis, timing analysis is based mainly on logic delays.

However, after place-and-route, routing delays, wire lengths, congestion, and FPGA interconnects significantly impact timing.

Therefore, the FPGA implementation provides a more accurate and realistic estimation of the processor's maximum operating frequency.

11.8 FPGA Utilization



The screenshot shows the 'Utilization' tab in a design tool. On the left, a 'Hierarchy' pane lists components: Slice LUTs (1%), LUT as Logic (1%), F8 Muxes (<1%), F7 Muxes (<1%), Slice Registers (1%), Register as Flip Flop, Slice Logic Distribution, and Slice (3%). The main table displays the following data:

Name	Slice LUTs (133800)	Slice Registers (267600)	F7 Muxes (66900)	F8 Muxes (33450)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	Bonded IOB (400)	BUFGCTRL (32)
top	892	1164	152	64	1096	892	69	19	1
decoder (decode_stage)	231	36	0	0	105	231	8	0	0
execute (execute_stage)	169	52	16	0	78	169	0	0	0
fetch (fetch_stage)	19	24	0	0	16	19	8	0	0
memory (memory_sta_)	473	1052	136	64	1040	473	16	0	0

Explanation

The FPGA utilization report summarizes the hardware resources consumed by the processor, including LUTs, flip-flops, and I/O pins.

The utilization confirms that the design fits comfortably within the target FPGA device.

11.9 Discussion

Although the design meets timing at the synthesis stage, timing violations appear after FPGA implementation due to routing delays.

This behavior is expected for complex pipelined designs and highlights the importance of implementation-aware timing analysis.

11.10 Limitations and Challenges

Despite the correct functional behavior of the processor, several challenges were encountered.

The most significant limitation is the timing degradation after FPGA implementation due to routing and clock distribution delays.

In addition, the control logic for interrupts, calls, and pipeline hazards increased the critical path length, limiting the maximum achievable frequency.

11.11 Future Work

Future improvements may include optimizing the pipeline stages to reduce the critical path, introducing clock gating techniques to reduce power consumption, and designing a dedicated interrupt controller.

Further enhancements such as instruction caching and improved hazard detection can significantly increase performance.

11.12 Conclusion

In this project, a complete pipelined processor was designed, implemented, and verified using Verilog HDL. The processor supports arithmetic, memory, control, and interrupt instructions.

Functional verification and synthesis results demonstrate correct operation, while timing analysis highlights the impact of physical implementation on performance.

11.13 Verification Summary

Functional verification was performed using a comprehensive testbench that covered arithmetic operations, memory access, control flow, and interrupt handling. Waveform analysis confirmed correct pipeline behavior and data forwarding.

Appendix

The video link

https://drive.google.com/file/d/1Y-I0XzXl_oo2uDIB8RmWDIS2D1UtsN9g/view?usp=drive_link